Manual del plc-bridge

Configurar y modificar el puente entre el PLC de Preparación y el dashboard

CONTENIDO

01 Qué es el bridge y cómo funciona

02 Archivos de la carpeta

03 El archivo de configuración `bridge.conf`

04 Pasar una variable de TIA Portal a dirección del bridge

05 Tareas frecuentes (paso a paso)

06 Usarlo en una PC de prueba

07 Usarlo en el servidor (Docker)

08 Qué datos envía el bridge

09 Diagnóstico de fallas

10 Qué cambios sí requieren programación

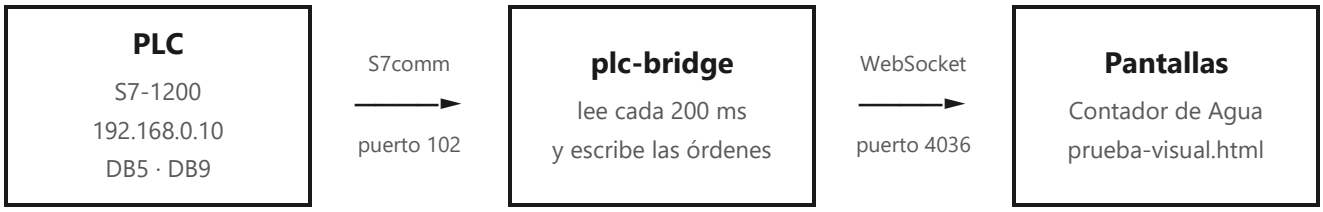
Versión 1.2 · 30 de octubre de 2026

PLC_1: S7-1200 CPU 1212C · proyecto «Prom_Preparacion» · lectura: DB5 (DB_MONITOREO) · órdenes: DB9 (DB_COMANDOS)

Las secciones marcadas AUTOMATIZACIÓN o SISTEMAS indican quién las usa normalmente.

1. Qué es el bridge y cómo funciona

El **plc-bridge** es un programa pequeño (hecho en Node.js) que lee los datos del PLC de Preparación y se los reenvía en tiempo real a las pantallas que los muestran: la pantalla «Contador de Agua» de la app y la página de prueba `prueba-visual.html`. También lleva al PLC las órdenes de la app para abrir y cerrar las **válvulas auxiliares**.



- **Del PLC al bridge:** el bridge se conecta al PLC por el protocolo S7comm (puerto 102). No requiere programa adicional en el PLC; solo que el bloque tenga el acceso optimizado desactivado y que la CPU permita PUT/GET.
- **Del bridge a las pantallas:** el bridge abre un servidor WebSocket (por defecto en el puerto 4036). Cada pantalla que se conecta recibe un mensaje con todos los valores, varias veces por segundo.
- **Sin cálculos:** el bridge pasa los valores tal cual los entrega el PLC (litros, caudal, total del tanque). No calcula caudales, totales ni tiempos.
- **Solo escribe las válvulas auxiliares:** lo único que escribe en el PLC son los bits `Valv_Aux_Tank1` … `3` del DB9 (DB_COMANDOS), cuando alguien con permiso abre o cierra una válvula auxiliar desde la app (ver «Control de las válvulas auxiliares»). No controla las válvulas de agua ni cambia setpoints.

Registro de llenados

Además de reenviar los datos, el bridge lleva un **registro de llenados**, que se ve en la pantalla «Contador de Agua» y en `prueba-visual.html`:

- Un llenado **empieza** cuando el PLC selecciona un tanque (`PLC_ID_Tanque` pasa a 1, 2 o 3).
- **Termina** cuando el PLC selecciona otro tanque o pasa a 0 (ninguno). Si pasa a 0 y vuelve al mismo tanque, quedan **dos** llenados.
- Por cada llenado se guarda el tanque, la hora de inicio, la hora de fin y el **total ingresado**: el último valor de `PLC_Totales_Litros` que se leyó mientras el tanque estuvo seleccionado.
- El PLC pone ese total en cero solo al cambiar de tanque. Por eso, si se pasa a 0 y se vuelve al mismo tanque, el segundo llenado muestra el total acumulado de los dos.
- Mientras no hay comunicación con el PLC no se abre ni se cierra ningún llenado. Si el bridge se reinicia con un llenado abierto, ese llenado se cierra con el motivo «bridge reiniciado».

- Las pantallas muestran los **últimos 20**. El archivo guarda hasta 500 (sección 7 para el servidor; en una PC queda en la carpeta `datos`).

Control de las válvulas auxiliares

Debajo de cada tanque, la pantalla «Contador de Agua» tiene un botón con el estado de su válvula auxiliar: **1** (abierta, verde) o **0** (cerrada), cuánto tiempo lleva abierta o cuándo se cerró, y quién lo hizo.

- **Quién la ve y quién la acciona:** la pantalla solo la ve el área Producción Aséptico. Abrir y cerrar pide el permiso «Abrir y cerrar las válvulas auxiliares (Contador de Agua)», que trae el rol **Supervisor**. A otra persona se le da como permiso extra desde **Personal** (sección 5.8).
- **Cómo se usa:** se toca el botón y la app pide confirmar («¿Seguro? Se va a abrir la válvula auxiliar del Tanque 2 en el PLC»). Con «Sí, abrir» la orden sale.
- **Qué pasa con la orden:**
  - la app la manda al bridge con la credencial de la sesión;
  - el bridge le pregunta a la base de datos si esa persona tiene el permiso; ahí la orden queda en la **Auditoría**;
  - con el permiso confirmado, el bridge escribe `Valv_Aux_TankN` en el DB9.
- **Lo que se ve es siempre lo que lee el PLC:** el 1/0 sale de `Actuador_Aux_Tank_N` (DB5), no de la orden. La pantalla muestra «Esperando al PLC…» y solo cambia cuando el PLC lo confirma. Si a los 10 s no cambió, avisa que se revise en la HMI.
- **La HMI escribe el mismo bit.** Abrir o cerrar desde la HMI también se ve en la app (con «por HMI»), y una orden de la app nunca queda «pegada».
- **Si se pierde la comunicación** entre el bridge y el PLC, el botón queda bloqueado y la válvula sigue como estaba. Se puede cerrar desde la HMI, y el PLC tiene temporizadores que la cierran sola si queda abierta demasiado tiempo.

Registro de válvulas auxiliares

- Cada vez que `Actuador_Aux_Tank_N` pasa a 1 se abre una apertura, y se cierra cuando vuelve a 0.
- Por cada apertura se guarda el tanque, cuándo se abrió, **quién la abrió**, cuándo se cerró, **quién la cerró** y el tiempo que estuvo abierta.
- **Quién:** el nombre de la persona si el cambio llegó dentro de los 10 s siguientes a su orden desde la app; «HMI» si no hubo orden de la app (la abrieron o cerraron desde la HMI o la lógica del PLC).
- Mientras no hay comunicación con el PLC no se registra nada. Si en ese rato la cierran desde la HMI, el cierre queda con la hora en que vuelve la comunicación.

- Igual que los llenados: si el bridge se reinicia con una apertura abierta, se cierra con «bridge reiniciado»; las pantallas muestran las **últimas 20** y el archivo guarda hasta 500.

Dos modos de trabajo

Modo	Qué hace	Cuándo se usa
`real`	Lee el PLC de verdad.	Con la PC o el servidor conectados a la red del PLC.
`simulacion`	Inventa datos de llenado (un tanque a la vez, setpoints al azar). La válvula auxiliar del tanque que termina de llenarse se abre 45 s, y las órdenes de la app funcionan. No necesita PLC.	Para probar las pantallas o la red sin tocar el PLC.

Dónde se configura todo

Toda la configuración está en **un solo archivo de texto**: `bridge.conf`, dentro de la carpeta `plc-bridge`. Ahí se define a qué PLC se conecta, qué variables lee y escribe, en qué puerto envía los datos y dónde está la base de datos que autoriza las órdenes. Para cambiar algo no hace falta tocar el código: se edita ese archivo y se reinicia el bridge.

2. Archivos de la carpeta

Archivo	Para qué sirve	¿Se edita?
`bridge.conf`	Toda la configuración: PLC, tags, puerto, modo.	**Sí**
`plc-bridge.bat`	Menú para la PC de prueba: 1) bridge con el PLC real, 2) bridge en simulación, 3) prueba de escritura manual.	No
`prueba-visual.html`	Página para ver los datos en el navegador (doble clic).	No
`src\`	El código del bridge (`index.js`, `config.js`, `control.js`, `registro.js`, `prueba-escritura.js`, `fuentes\s7.js`, `fuentes\simulacion.js`).	Solo programación
`package.json`	Lista de librerías que usa (`nodes7` y `ws`).	No
`node_modules\`	Las librerías ya instaladas (lo crea `npm install`).	No
`Dockerfile`	Receta para correr el bridge en el servidor con Docker.	No

Archivo	Para qué sirve	¿Se edita?
`datos\`	Registros de llenados (`llenados.json`) y de válvulas auxiliares (`aperturas-aux.json`); en simulación, con `-simulacion` en el nombre. Los crea el bridge.	No
`Manual-plc-bridge.docx`	Este manual (editable). También hay una versión PDF.	Sí, para actualizarlo

3. El archivo de configuración bridge.conf

Es un archivo de texto plano. Se abre con el **Bloc de notas** (clic derecho → Abrir con → Bloc de notas). Tiene seis partes:

```
# 1. Modo de trabajo
PLC_MODO=real

# 2. A dónde envía la información
PLC_BRIDGE_PUERTO=4036
PLC_BRIDGE_HOST=0.0.0.0
PLC_BRIDGE_INTERVALO_MS=200

# 3. De dónde lee la información (PLC de origen)
PLC_HOST=192.168.0.10
PLC_PUERTO=102
PLC_RACK=0
PLC_SLOT=1

# 4. Tags base (los usa la pantalla para dibujar los tanques)
PLC_TAG_SETPOINT=DB5,DINT0
PLC_TAG_VALVULA_1=DB5,X14.0
# ...

# 5. Control remoto de las válvulas auxiliares (DB9)
PLC_TAG_CMD_AUX_1=DB9,X0.0
# ...
PLC_API_URL=http://192.168.10.143:4037

# 6. Tags adicionales (libres)
PLC_EXTRA_TEMPERATURA_AGUA=DB5,REAL16
```

Reglas para escribirlo

- **Una línea por ajuste**, con la forma `CLAVE=VALOR`.
- **Sin espacios** alrededor del `=` y sin comillas.

- Las líneas que empiezan con `#` son **comentarios**: el bridge las ignora. Para "apagar" un ajuste sin borrarlo, ponle `#` adelante.
- Las claves van en **MAYÚSCULAS**, con números y guion bajo (_). Nada de tildes, eñes ni espacios.
- Una clave vacía (`PLC_REGISTRO_CARPETA=`) equivale a no definirla: se usa el valor por defecto. En un tag base, vacío = ese dato llega como «sin dato».

> **ATENCIÓN** **Después de cualquier cambio hay que reiniciar el bridge.**
> El bridge lee `bridge.conf` solo al arrancar. En la PC de prueba: detenerlo (`Ctrl+C`, responder `N`) y volver a elegir la opción en `plc-bridge.bat`. En el servidor: ver la sección 7.

Todas las claves

Clave	Qué es	Valor actual
`PLC_MODO`	`real` o `simulacion`.	real
`PLC_BRIDGE_PUERTO`	Puerto donde el bridge espera a las pantallas.	4036
`PLC_BRIDGE_HOST`	Interfaz de red donde escucha. `0.0.0.0` = todas (cualquier PC de la red se puede conectar).	0.0.0.0
`PLC_BRIDGE_INTERVALO_MS`	Cada cuántos milisegundos lee el PLC y envía los datos. Recomendado entre 100 y 500.	200
`PLC_REGISTRO_CARPETA`	Carpeta donde se guardan los registros de llenados y de válvulas auxiliares. Vacío = carpeta `datos` del bridge.	(vacío)
`PLC_HOST`	IP del PLC.	192.168.0.10
`PLC_PUERTO`	Puerto S7comm del PLC (casi siempre 102).	102
`PLC_RACK` / `PLC_SLOT`	Posición de la CPU. S7-1200/1500: rack 0, slot 1. S7-300/400: rack 0, slot 2.	0 / 1
`PLC_TIMEOUT_MS`	Cuánto espera una respuesta del PLC antes de darla por fallida.	5000
`PLC_RECONEXION_MS`	Cada cuánto reintenta si se pierde la conexión.	5000
`PLC_TAG_SETPOINT`	Setpoint de litros que puso el operador.	DB5,DINT0
`PLC_TAG_LITROS_ACTUALES`	Litros que van en la apertura en curso.	DB5,DINT4

Clave	Qué es	Valor actual
`PLC_TAG_TOTAL_TANQUE`	`PLC_Totales_Litros`: total que entró al tanque seleccionado. El PLC lo suma y lo pone en cero al cambiar de tanque. Es el valor que guarda el registro.	DB5,DINT8
`PLC_TAG_TANQUE_ACTIVO`	Tanque en uso: 0 = ninguno, 1, 2 o 3.	DB5,INT12
`PLC_TAG_VALVULA_1` … `_3`	Válvula de entrada de agua de cada tanque.	DB5,X14.0 … X14.2
`PLC_TAG_AUX_1` … `_3`	`Actuador_Aux_Tank_1` … `3`: estado de la válvula auxiliar de cada tanque (recirculación / CIP). Es el 1/0 que muestra el botón y el que alimenta el registro.	DB5,X14.3 … X14.5
`PLC_TAG_CAUDAL`	`PLC_Caudal_LPM`: caudal del flujómetro en L/min, calculado por el PLC.	DB5,INT16
`PLC_TAG_CMD_AUX_1` … `_3`	`Valv_Aux_Tank1` … `3` (DB9): el bit que escribe el bridge para abrir (1) o cerrar (0) la válvula auxiliar. La HMI escribe el mismo bit.	DB9,X0.0 … X0.2
`PLC_API_URL`	API de la base de datos, donde el bridge revisa el permiso de quien da la orden y deja la auditoría. Vacío = control de válvulas apagado. En el servidor, `docker-compose.yml` la cambia por `http://api:3000`.	http://192.168.10.143:4037
`PLC_EXTRA_<NOMBRE>`	Tags adicionales libres (ver sección 5.2).	(ninguno)

> **NOTA** **Prioridad: el .bat y el servidor pueden pisar un valor.**
>
> Si una clave también está definida en el `.bat` (con `set`) o en `docker-compose.yml` (en `environment:`), **gana esa** y lo que diga `bridge.conf` se ignora. Hoy pasa con dos claves: `PLC_MODO` (`plc-bridge.bat` fuerza `real` en la opción 1 y `simulacion` en la 2; el servidor fuerza `real`) y `PLC_API_URL` (el servidor usa `http://api:3000`).

4. Pasar una variable de TIA Portal a dirección del bridge

AUTOMATIZACIÓN

Requisitos en el PLC (una sola vez)

1. **Acceso optimizado desactivado** en el DB: clic derecho sobre el DB → Propiedades → Atributos → desmarcar «Acceso optimizado al bloque». Solo así aparece la columna **Offset**.

2. **PUT/GET permitido** en la CPU: Propiedades de la CPU → Protección y seguridad → Mecanismos de conexión → «Permitir acceso vía comunicación PUT/GET del interlocutor remoto».
3. **Compilar y cargar** el DB después de cualquier cambio. Los offsets que muestra TIA solo son válidos después de compilar.
4. Para el **DB9 (DB_COMANDOS)**, donde escribe el bridge: lo mismo (acceso optimizado desactivado) y las variables `Valv_Aux_Tank1` … `3` marcadas como escribibles.

Formato de la dirección

La dirección siempre tiene la forma `DB<número>,<TIPO><offset>`. De TIA Portal se necesitan tres datos: el **número del DB**, el **Data type** y el **Offset** de la variable.

Tipo en TIA	Se escribe	Offset en TIA	Dirección resultante
Bool	`X` + byte.bit	14.3	`DB5,X14.3`
Byte	`BYTE`	16.0	`DB5,BYTE16`
Char	`CHAR`	17.0	`DB5,CHAR17`
Int	`INT`	12.0	`DB5,INT12`
Word	`WORD`	12.0	`DB5,WORD12`
DInt	`DINT`	0.0	`DB5,DINT0`
DWord	`DWORD`	0.0	`DB5,DWORD0`
Real	`REAL`	4.0	`DB5,REAL4`

- En todos los tipos **menos Bool**, del offset se usa solo la parte entera: `4.0` → `4`.
- En **Bool** se usan las dos partes: `14.3` → `X14.3` (byte 14, bit 3). El bit va de 0 a 7.
- Se puede escribir en minúsculas (`db5,real4`); el bridge lo convierte solo.
- No se admiten otros tipos (String, LReal, Time, arreglos…). Si hace falta uno, ver la sección 10.

> **ATENCIÓN** **El error más común: el tipo equivocado.**
>
> Si en TIA la variable es `Real` y en `bridge.conf` se escribe `DINT` (o al revés), el bridge lee los mismos bytes pero los interpreta mal: aparecen números absurdos, como millones o negativos. Antes de revisar cualquier otra cosa, confirma que el tipo coincide.

Ejemplo completo

Automatización agrega al DB5 una variable `Temp_Agua`, tipo **Real**, que después de compilar queda en el offset **16.0**. La línea para `bridge.conf` es:

```
PLC_EXTRA_TEMP_AGUA=DB5,REAL16
```

5. Tareas frecuentes (paso a paso)

5.1 Corregir la dirección de un tag que ya existe AUTOMATIZACIÓN

Por ejemplo, porque se reorganizó el DB5 y el setpoint pasó del offset 0 al 20.

1. En TIA Portal, compila el DB y anota el nuevo offset y el tipo (sección 4).
2. Abre `bridge.conf` y busca la clave en la parte 4 (tags base). Ejemplo: `PLC_TAG_SETPOINT=DB5,DINT0`.
3. Cambia **solo la dirección**, no el nombre de la clave: `PLC_TAG_SETPOINT=DB5,DINT20`.
4. Guarda y reinicia el bridge.
5. Comprueba en `prueba-visual.html` que el valor coincide con la HMI o con TIA en línea («Observar todo» sobre el DB).

> NOTA **No renombres las claves PLC_TAG_...**
> La pantalla las usa por su nombre para dibujar los tanques. Si cambias `PLC_TAG_SETPOINT` por otro nombre, el bridge deja de leer el setpoint.

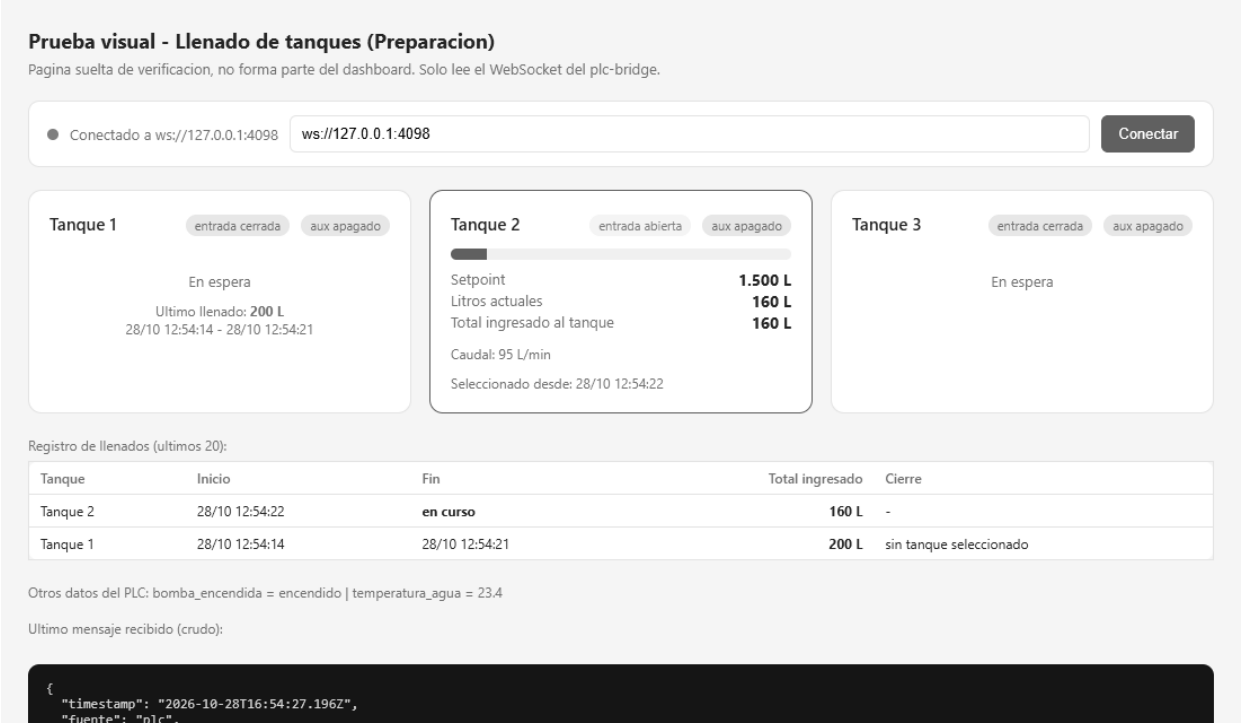
5.2 Agregar un tag nuevo AUTOMATIZACIÓN

1. Ten la variable creada y compilada en el PLC, y anota DB, tipo y offset.
2. Elige un nombre corto y claro, en MAYÚSCULAS, con _ en lugar de espacios. Ejemplo: `PRESION_LINEA`.
3. Al final de `bridge.conf` (parte 5) agrega una línea que empiece con `PLC_EXTRA_` + el nombre:

   ```
   PLC_EXTRA_PRESION_LINEA=DB5,REAL24
   ```

4. Guarda y reinicia el bridge. Al arrancar, la ventana lista todos los tags que va a leer. Ahí debe aparecer `extra.presion_linea  DB5,REAL24`.
5. Comprueba el valor (los Bool se muestran como Encendido/Apagado; los números, tal cual):
   - en `prueba-visual.html`, en la línea «Otros datos del PLC»;
   - en el dashboard, en el recuadro «Otros datos del PLC».

> **NOTA** **Agrega los tags de a uno.**
>
> Si un tag nuevo tiene una dirección que el PLC no puede entregar (DB que no existe, offset más allá del tamaño del DB), el resto sigue funcionando. Ese tag llega vacío («sin dato») y la consola y las pantallas muestran el aviso *«El PLC no entrega …»*. Agregándolos de a uno sabes enseguida cuál está mal.



prueba-visual.html con el tanque 2 seleccionado, el registro de llenados y dos tags adicionales (PLC_EXTRA_BOMBA_ENCENDIDA y PLC_EXTRA_TEMPERATURA_AGUA) en «Otros datos del PLC».

5.3 Quitar un tag adicional

Pon `#` al principio de su línea (o bórrala) y reinicia el bridge. Los tags base (`PLC_TAG_…`) no se quitan. Si alguno no existe en el PLC, escríbele una dirección válida o déjalo vacío; vacío, ese dato se muestra como «sin dato».

5.4 Cambiar el PLC de origen AUTOMATIZACIÓN

Por ejemplo, si el PLC cambia de IP o se reemplaza la CPU.

1. En la parte 3 de `bridge.conf`, cambia `PLC_HOST` por la nueva IP.
2. Si cambió el modelo de CPU, ajusta `PLC_RACK` y `PLC_SLOT` (S7-1200/1500: 0 y 1; S7-300/400: 0 y 2).
3. Si los datos ahora están en otro DB, cambia el número en **todas** las direcciones (por ejemplo, `DB5,` → `DB12,`) y revisa los offsets.

4 Verifica que la PC o el servidor estén en la misma red que la nueva IP, o que haya una ruta hasta ella (sección 9).

5 Guarda, reinicia y comprueba.

5.5 Cambiar a dónde envía la información (IP / puerto) SISTEMAS

El bridge no "empuja" datos hacia una IP: **abre un puerto y las pantallas se conectan a él**. Por eso, cambiar a dónde llega la información tiene dos partes.

A) Lado del bridge, en `bridge.conf`:

- `PLC_BRIDGE_PUERTO` es el puerto en el que escucha (por ejemplo, `4040`).
- `PLC_BRIDGE_HOST` se deja en `0.0.0.0`. Solo se cambia para limitar por qué tarjeta de red acepta conexiones (por ejemplo, `127.0.0.1` = solo la misma PC).

B) Lado de quien lo muestra:

Pantalla	Qué cambiar
`prueba-visual.html`	Escribe la nueva dirección en el campo de arriba y pulsa **Conectar**. Formato: `ws://<IP de la PC o servidor del bridge>:<puerto>`. Ejemplo: `ws://192.168.10.143:4036`.
Dashboard (servidor)	• La dirección está en la variable `VITE_PLC_BRIDGE_WS_URL` del archivo `.env.local` del proyecto. Si no está definida, usa `ws://192.168.10.143:4036`. • Esta variable queda fija **al construir** la app: después de cambiarla hay que reconstruir el servicio `preview` (sección 7).

> **ATENCIÓN** **En el servidor, el puerto aparece en dos lugares.**
>
> En `docker-compose.yml`, servicio `plc-bridge`, la línea `ports: - "4036:4036"`:
>
> – el **primer** número es el puerto que se abre en el servidor;
>
> – el **segundo** tiene que ser igual a `PLC_BRIDGE_PUERTO`.
>
> Si cambias el puerto en `bridge.conf`, cambia también esa línea.

5.6 Cambiar la frecuencia de lectura

`PLC_BRIDGE_INTERVALO_MS`: 200 = 5 lecturas por segundo. Valores recomendados: entre 100 y 500.

- **Más bajo:** se ve más fluido, pero carga más al PLC y a la red.
- **Más alto:** descansa al PLC, pero la pantalla se ve "a saltos".

5.7 Probar la escritura al PLC sin la app AUTOMATIZACIÓN

Sirve para comprobar que el PLC acepta las escrituras al DB9, por ejemplo después de cambiar el DB o la CPU. La IP del PLC y las direcciones salen de `bridge.conf`.

> **ATENCIÓN Esta prueba mueve la válvula de verdad.**
> Escribir 1 abre la válvula auxiliar de ese tanque y 0 la cierra. Hazla con el área avisada.

1. Doble clic en `plc-bridge.bat` y elige **3**.
2. La consola muestra el valor actual de `Valv_Aux_Tank1` … `3`.
3. Elige la variable (1, 2 o 3), escribe `1` o `0` y confirma con `s`.
4. La consola lee de vuelta y dice si el PLC tiene el valor. Comprueba que la HMI muestra lo mismo.
5. Con `0` vuelves al menú.

5.8 Dar permiso para accionar las válvulas SISTEMAS

Los supervisores ya lo tienen por su rol. Para dárselo a otra persona: en **Personal**, abre la tarjeta de la persona, ve a los permisos extra y marca «Abrir y cerrar las válvulas auxiliares (Contador de Agua)». La persona lo tiene la próxima vez que abra la app.

6. Usarlo en una PC de prueba AUTOMATIZACIÓN

Preparar la PC (una sola vez)

1. Instala **Node.js** (versión LTS) desde nodejs.org. Para comprobarlo, en una ventana de comandos (`cmd`) escribe `node --version`: debe responder con un número de versión.
2. Copia la carpeta `plc-bridge` completa a la PC.
3. Si no copiaste `node_modules`, abre `cmd` dentro de la carpeta y ejecuta:

```
npm install
```

4. Si se conecta directo al PLC con un cable, ponle a la tarjeta de red una **IP fija de la red del PLC**. Por ejemplo, `192.168.0.20`, máscara `255.255.255.0`. Que no se repita con otro equipo.
5. Comprueba que hay comunicación: `ping 192.168.0.10` debe responder.

Correrlo

1. Doble clic en `plc-bridge.bat` y elige **1** (PLC real) o **2** (simulación, sin PLC). Si es la primera vez y falta `node_modules`, el `.bat` instala las librerías solo.
2. Revisa lo que muestra la ventana al arrancar. Debe verse algo así:

```
[plc-bridge] configuracion: C:\...\plc-bridge\bridge.conf
[plc-bridge] PLC 192.168.0.10:102 rack 0 slot 1
[plc-bridge]   setpoint_l                  DB5,DINT0
[plc-bridge]   litros_actuales             DB5,DINT4
[plc-bridge]   ...
[plc-bridge]   extra.presion_linea         DB5,REAL24
[plc-bridge]   cmd_aux_1                   DB9,X0.0 (escritura)
[plc-bridge]   ...
[plc-bridge] control de valvulas auxiliares habilitado (base: http://192.168.10.143:4037)
[plc-bridge] modo=real puerto=4036 intervalo=200ms
[plc-bridge] escuchando en 0.0.0.0:4036
```

3. Abre `prueba-visual.html` con doble clic. Se conecta solo a `ws://localhost:4036`.
4. Para ver los datos desde **otra PC** de la red: abre ahí la misma página y escribe `ws://<IP de la PC del bridge>:4036`.
5. Para detenerlo: pulsa `Ctrl+C` y responde `N` para volver al menú (`S` cierra la ventana).

Probar el control de las válvulas desde la PC de prueba

Para leer basta con lo anterior. Para **abrir y cerrar** desde la app hace falta además:

1. **Llegar a las dos redes a la vez:** al PLC (`192.168.0.x`) y al servidor `192.168.10.143`, que tiene la base de datos donde se entra a la app y se revisa el permiso. Lo práctico: el cable al switch del PLC más el Wi-Fi o un segundo cable a la red de planta. Comprueba `ping 192.168.0.10` y `ping 192.168.10.143`.
2. **La base de datos con la migración** `20261113490000_valvula_aux_remota.sql` aplicada (el permiso y la autorización viven ahí).
3. Arranca el bridge con `plc-bridge.bat`, opción **1**. Debe decir «control de valvulas auxiliares habilitado».
4. Con el proyecto completo en la PC, abre la app conectada a **este** bridge (PowerShell, en la carpeta del proyecto):

```
$env:VITE_PLC_BRIDGE_WS_URL = "ws://localhost:4036"
npm run dev
```

5. Entra a `http://localhost:5173` con un usuario Supervisor (o con el permiso extra) y prueba desde «Contador de Agua».

> **NOTA** **No pongas VITE_PLC_BRIDGE_WS_URL en .env.local para esta prueba.**
>
> Esa carpeta se sincroniza con el servidor y la dirección terminaría en la app de planta. El registro de esta prueba queda en la PC (`plc-bridge\datos\`); la auditoría de cada orden, en la base de datos.

> **NOTA** **Firewall de Windows.**
>
> La primera vez, Windows puede preguntar si permite a "Node.js" usar la red. Acepta en **redes privadas**; si no, las otras PCs no podrán conectarse al puerto 4036.

7. Usarlo en el servidor (Docker) SISTEMAS

En el servidor de planta (`192.168.10.143`) el bridge corre como un contenedor Docker llamado `plc-bridge`, junto con el resto de la app. El proyecto está en `/home/inf05/natulac-aseptico`.

Cómo está montado

- Los registros se guardan **fuera del proyecto**, en `/home/inf05/natulac-plc-datos/` (`llenados.json` y `aperturas-aux.json`), para que subir archivos por WinSCP o reconstruir el contenedor no los borre. Para empezar un registro nuevo: detener el bridge, borrar ese archivo y volver a arrancarlo.
- El contenedor lee el `bridge.conf` del proyecto: `docker-compose.yml` lo monta directo en `/app/bridge.conf`.
- Por eso, un cambio en `bridge.conf` **solo requiere reiniciar** el contenedor, no reconstruirlo.
- `docker-compose.yml` fuerza `PLC_MODO=real` y `PLC_API_URL=http://api:3000` (la API de la base de datos dentro de Docker).
- El control de las válvulas necesita que la base de datos tenga la migración `20261113490000_valvula_aux_remota.sql` (se aplica con `bash scripts/aplicar-migracion.sh`).

Cambiar la configuración en el servidor

1. Edita `plc-bridge/bridge.conf` en tu PC y súbelo por WinSCP a `/home/inf05/natulac-aseptico/plc-bridge/`.
2. Entra por SSH (`ssh inf05@192.168.10.143`) y reinicia solo el bridge:

```
cd natulac-aseptico
docker compose --env-file .env.local --profile preview restart plc-bridge
```

3. Revisa que arrancó bien y qué tags cargó:

```
docker compose --env-file .env.local --profile preview logs --tail 40 plc-bridge
```

> **ATENCIÓN Siempre con --env-file .env.local.**
>
> `docker-compose.yml` exige los datos de la base de datos (DB_HOST, etc.) que están en `.env.local`. Sin esa opción, Docker se niega a ejecutar cualquier comando, aunque solo se quiera reiniciar el bridge.

Cuándo hay que reconstruir (no basta reiniciar)

Qué cambió	Comando
Solo `bridge.conf`	`... restart plc-bridge` (arriba)
El código del bridge (`src\`), `package.json` o `Dockerfile`	`docker compose --env-file .env.local --profile preview up -d --build plc-bridge`
El puerto en `docker-compose.yml` o `PLC_MODO` del servidor	`docker compose --env-file .env.local --profile preview up -d plc-bridge`
`VITE_PLC_BRIDGE_WS_URL` (dirección que usa el dashboard)	`docker compose --env-file .env.local --profile preview up -d --build preview` — *reinicia el dashboard: hacerlo con poca gente usándolo.*

Comunicación del servidor con el PLC

El servidor ya corre el bridge en modo real, pero el PLC está en la red `192.168.0.x` y el servidor en la `192.168.10.x`: hace falta una ruta entre ambas. Sin ella, la pantalla muestra que no hay comunicación con el PLC y el botón de las válvulas queda bloqueado. Para comprobarla, desde el servidor (las dos pruebas deben responder):

```
ping -c 3 192.168.0.10
nc -zv 192.168.0.10 102
```

> **NOTA Un solo bridge conectado al PLC.**
>
> Si el servidor y una PC de prueba corren el bridge en modo real al mismo tiempo, los dos leen y aceptan órdenes. Mientras se prueba en la PC, conviene que el del servidor no llegue al PLC (o detenerlo).

8. Qué datos envía el bridge SISTEMAS

Cada `PLC_BRIDGE_INTERVALO_MS` milisegundos, el bridge manda un mensaje JSON a cada pantalla conectada. Las pantallas que se conectan reciben enseguida el último mensaje. Ese mismo mensaje se ve en `prueba-visual.html` (recuadro oscuro) y en el dashboard («Ver último mensaje crudo»).

Campo	Contenido
`timestamp`	Fecha y hora de la lectura (UTC).
`fuente`	`plc` o `simulacion`.
`conectado`	Solo en modo real: `true` si la última lectura del PLC fue válida.
`tanque_activo`	1, 2, 3 o `null` (ninguno).
`setpoint_l`	Setpoint de litros.
`litros_actuales`	Litros de la apertura en curso.
`litros_tanque`	Total que entró al tanque seleccionado (`PLC_Totales_Litros`).
`caudal_l_min`	Caudal en L/min (`PLC_Caudal_LPM`).
`valvulas`	Válvula de entrada de cada tanque: `{"1": true, "2": false, "3": false}`.
`auxiliares`	Válvula auxiliar (recirculación/CIP) de cada tanque, mismo formato.
`llenando`	`true` si alguna válvula de entrada está abierta.
`extra`	Los tags `PLC_EXTRA_...`, con el nombre en minúsculas: `{"presion_linea": 2.35}`. `null` = el PLC no entregó ese tag.
`tags_con_error`	Lista de tags que el PLC no pudo entregar en esta lectura (dirección mal configurada).
`llenados`	Últimos 20 llenados, el más reciente primero: `tanque`, `inicio`, `fin` (`null` = en curso), `litros_tanque` y `motivo_fin`. Horas en UTC; las pantallas las muestran en hora de planta.
`aperturas_aux`	Últimas 20 aperturas de las válvulas auxiliares: `tanque`, `inicio`, `abierta_por`, `fin` (`null` = sigue abierta), `cerrada_por` y `motivo_fin`. Quién: usuario y nombre si fue una orden de la app, `null` si fue desde la HMI.
`control_aux`	`true` si el bridge acepta órdenes para las válvulas auxiliares (`PLC_API_URL` definida).

Órdenes de la app al bridge

La pantalla manda por el mismo WebSocket:

```
{ "tipo": "valvula_aux", "id": "orden-1", "tanque": 2, "abrir": true,
  "usuario": "<usuario>", "sesion": "<credencial de la sesión>" }
```

y el bridge contesta `{ "tipo": "respuesta", "id": "orden-1", "ok": true }`, o `ok: false` con `error` (el texto que muestra la pantalla). Antes de escribir, el bridge llama a `autorizar_valvula_aux` en la base de datos con esa credencial: sin permiso, no escribe nada.

Cuando no hay comunicación con el PLC, el bridge envía el mensaje con `conectado: false` y los valores en `null`, para que ninguna pantalla muestre datos viejos como si fueran actuales.

9. Diagnóstico de fallas

Qué se ve	Causa probable	Qué hacer
`'node' no se reconoce como un comando…`	Node.js no está instalado.	Instalar Node.js LTS y volver a abrir el `.bat`.
`Cannot find package 'ws'` o `'nodes7'`	Faltan las librerías.	Ejecutar `npm install` dentro de la carpeta `plc-bridge`.
`ERROR en la configuracion: … no es una direccion valida`	Dirección mal escrita en `bridge.conf`.	Revisar el formato `DB5,TIPOoffset` (sección 4). El mensaje dice qué clave es.
`ERROR en la configuracion: bridge.conf linea N: falta el "="`	Línea incompleta o con texto suelto.	Corregir esa línea o comentarla con `#`.
`la clave "…" solo puede tener MAYUSCULAS…`	Clave con minúsculas, tildes o espacios.	Escribirla en mayúsculas y con `_`.
`PLC_HOST no esta definido`	Modo real sin IP del PLC.	Completar `PLC_HOST` en `bridge.conf`.
Pantallas en «sin datos» o `conectado: false` todo el tiempo	No hay comunicación con el PLC.	• Revisar el cable y la IP de la PC (red `192.168.0.x`). • `ping 192.168.0.10`. • Confirmar `PLC_HOST`, rack y slot. • En el servidor: la ruta hasta 192.168.0.x (sección 7).
Conecta, pero **todos** los valores llegan vacíos	PUT/GET deshabilitado o DB con acceso optimizado.	Revisar los requisitos de la sección 4 en TIA Portal, compilar y cargar.
Aviso `El PLC no entrega "…"`	Ese tag apunta a un DB que no existe o a un offset fuera del tamaño del DB.	Revisar número de DB, offset y tipo de ese tag. El resto de los tags sigue funcionando.
Números absurdos (millones, negativos, decimales raros)	Tipo equivocado (por ejemplo, `DINT` en lugar de `REAL`) u offset corrido.	Comparar tipo y offset con TIA Portal.

Qué se ve	Causa probable	Qué hacer
Un Bool no cambia nunca, o cambia cuando cambia otro	Bit equivocado (`X14.0` en lugar de `X14.1`).	Revisar byte y bit del offset en TIA.
Cambié `bridge.conf` y no pasa nada	• No se reinició. • La clave está pisada por `plc-bridge.bat` o por `docker-compose.yml`. • En el servidor: no se subió el archivo.	Reiniciar y revisar la primera línea que muestra el bridge al arrancar (qué archivo usa). Ver «Prioridad» en la sección 3.
`listen EADDRINUSE … 4036`	El puerto ya está ocupado (otro bridge abierto).	Cerrar la otra ventana del bridge, o cambiar `PLC_BRIDGE_PUERTO`.
La página dice «Desconectado, reintentando…»	El bridge no está corriendo, la dirección es otra o el firewall la bloquea.	Confirmar que el bridge está corriendo, la IP/puerto del campo de arriba y el firewall de Windows (sección 6).
Botón de la válvula bloqueado: «Control remoto no disponible»	Sin comunicación con el PLC, o `PLC_API_URL` vacía (al arrancar, el bridge dice «control de valvulas auxiliares APAGADO»).	Revisar la comunicación con el PLC y `PLC_API_URL`.
Botón bloqueado: «No tienes permiso…»	La sesión no tiene el permiso de las válvulas auxiliares.	Darlo desde Personal (sección 5.8) y volver a abrir la app.
«El bridge no pudo consultar la base de datos»	El bridge no llega a `PLC_API_URL` (en una PC de prueba: falta la conexión a la red de planta).	Comprobar `ping 192.168.10.143` y la dirección en `bridge.conf`.
Error que menciona `autorizar_valvula_aux`	La base de datos no tiene la migración del control.	Aplicar `20261113490000_valvula_aux_remota.sql` (sección 7).
«El PLC rechazó la escritura»	DB9 con acceso optimizado, variable no escribible o PUT/GET deshabilitado.	Revisar los requisitos de la sección 4 y probar con la opción 3 de `plc-bridge.bat` (sección 5.7).
«El PLC no mostró el cambio»	El PLC aceptó el bit pero `Actuador_Aux_Tank_N` no cambió (lógica del PLC, enclavamiento o temporizador).	Revisar la válvula en la HMI y en TIA en línea.

Qué se ve	Causa probable	Qué hacer
El tag nuevo aparece en «Otros datos» pero no en el dibujo de los tanques	Es lo esperado.	Los tags adicionales se muestran como lista. Para que un dato se use en el dibujo hace falta programación (sección 10).

10. Qué cambios sí requieren programación

Con `bridge.conf` se resuelve: cambiar direcciones, agregar o quitar tags adicionales, cambiar de PLC, cambiar puerto y frecuencia. Lo siguiente **no** se puede hacer solo con la configuración y hay que pedírselo a quien mantiene el código:

- **Darle un significado nuevo a un tag en la pantalla.** Por ejemplo, que un nivel de tanque mueva la animación, o que un tag cambie de color según un rango. Los tags adicionales solo se listan.
- **Tipos de dato no admitidos:** String, LReal, Time, Date, arreglos o estructuras (UDT).
- **Más tanques o válvulas** que los tres de hoy.
- **Enviar los datos a otro destino**, como guardarlos en la base de datos o mandarlos a otro sistema.
- **Escribir otras variables en el PLC** (válvulas de agua, setpoint, arranque/parada). Hoy solo se escriben las válvulas auxiliares.

Para esos casos, el código está en `plc-bridge\src\`:

- `config.js`: lectura de `bridge.conf`.
- `fuentes\s7.js`: lectura del PLC y armado del mensaje.
- `index.js`: servidor WebSocket.
- `registro.js`: registros de llenados y de válvulas auxiliares.
- `control.js`: órdenes de la app (autorización en la base de datos y escritura al DB9).
- `prueba-escritura.js`: la prueba manual de la opción 3.
- La pantalla «Contador de Agua» está en `src\pages\apps\PreparacionPLC.tsx`, en la raíz del proyecto; el permiso y la autorización, en la migración `20261113490000_valvula_aux_remota.sql`.